```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

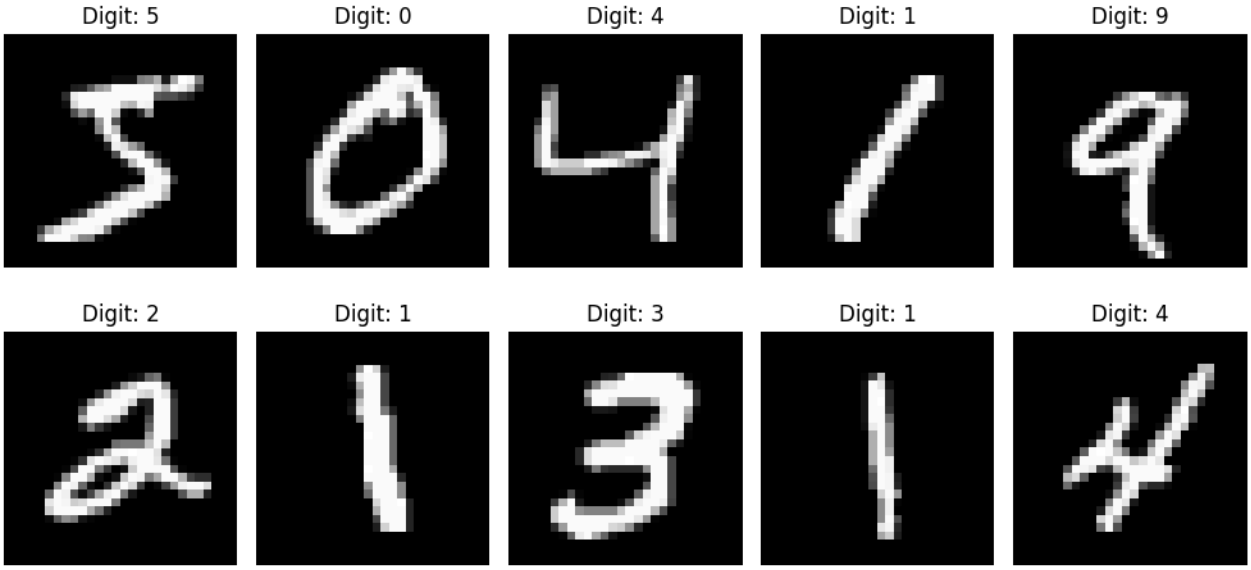
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Digit: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Normalize pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Training shape after preprocessing:", x_train.shape)
print("Testing shape after preprocessing:", x_test.shape)
```

```
Training shape after preprocessing: (60000, 28, 28, 1)
Testing shape after preprocessing: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
```

```
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(128, activation="relu"),

    layers.Dense(10, activation="softmax")
])

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully.")
```

```
Model compiled successfully.
```

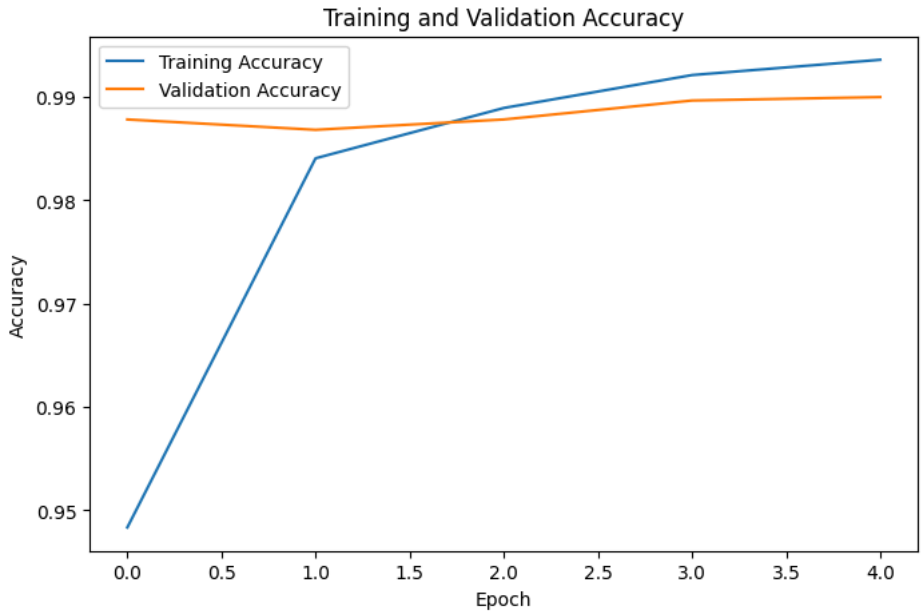
```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)
```

```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 47s 54ms/step - accuracy: 0.9484 - loss: 0.1740 - val_accuracy: 0.9878 - val_loss: 0.0487
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 45s 54ms/step - accuracy: 0.9841 - loss: 0.0501 - val_accuracy: 0.9868 - val_loss: 0.0521
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 53ms/step - accuracy: 0.9889 - loss: 0.0347 - val_accuracy: 0.9878 - val_loss: 0.0404
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 44s 53ms/step - accuracy: 0.9921 - loss: 0.0254 - val_accuracy: 0.9897 - val_loss: 0.0334
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 46s 54ms/step - accuracy: 0.9936 - loss: 0.0200 - val_accuracy: 0.9900 - val_loss: 0.0359
```

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

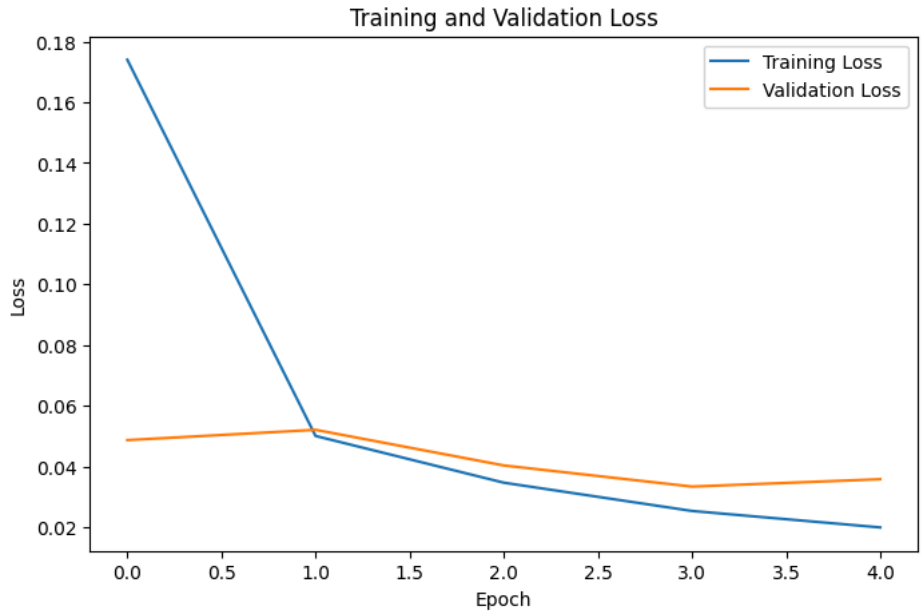
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.show()
```




```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.show()
```




```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

```
Test Loss: 0.0282
Test Accuracy: 0.9908
Test Accuracy: 99.08%
```

```
predictions = model.predict(x_test)

predicted_labels = np.argmax(predictions, axis=1)

print("Predictions generated successfully.")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step
Predictions generated successfully.
```

```
plt.figure(figsize=(12, 6))

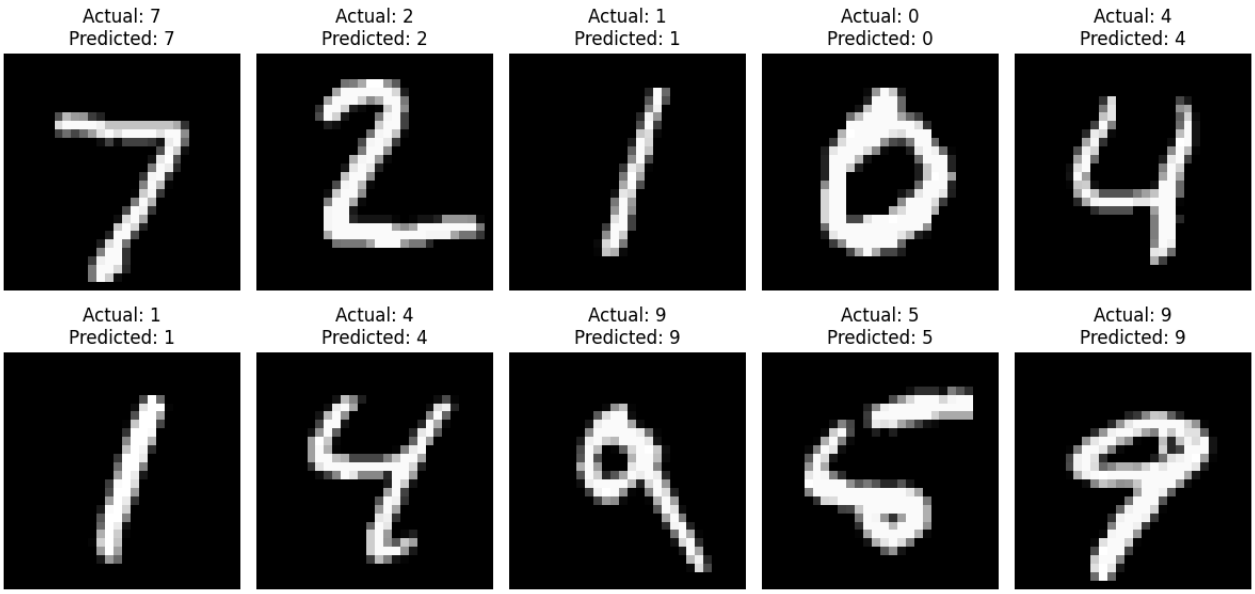
for i in range(10):
    plt.subplot(2, 5, i + 1)

    plt.imshow(x_test[i].squeeze(), cmap="gray")

    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {predicted_labels[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

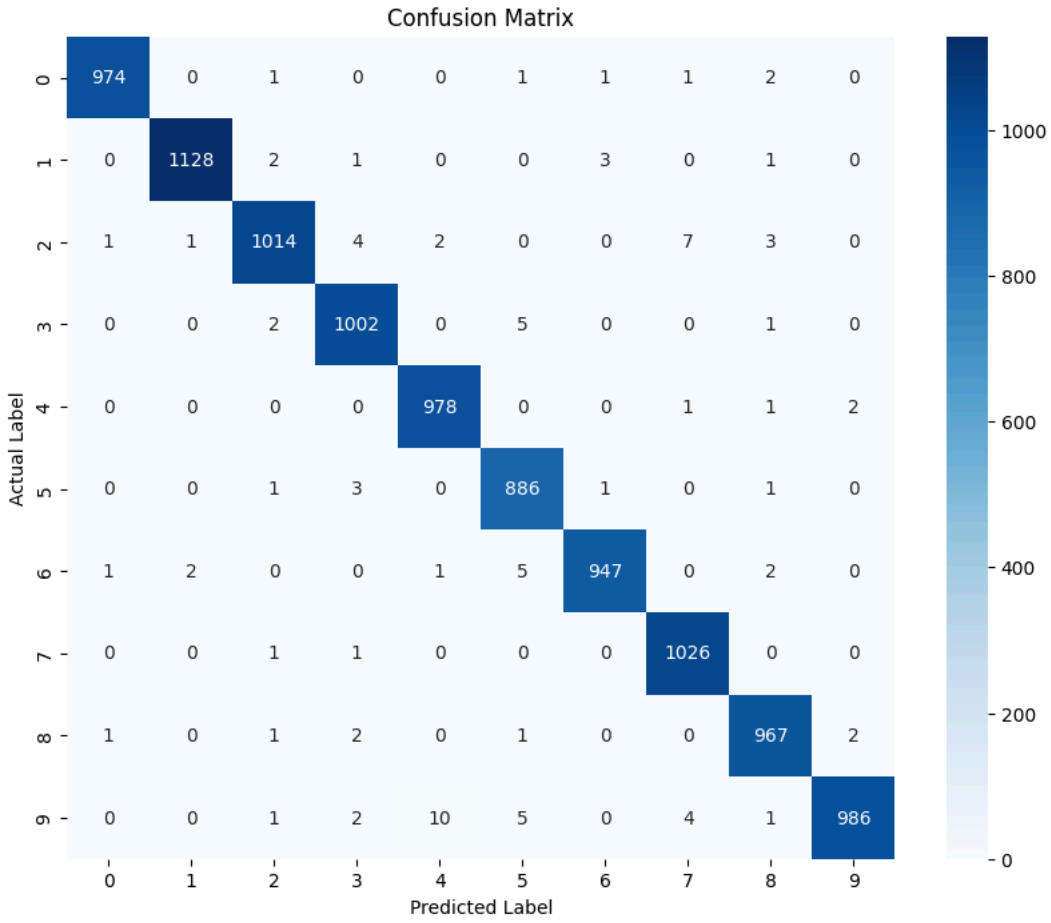



```
cm = confusion_matrix(y_test, predicted_labels)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=range(10),
    yticklabels=range(10)
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")
plt.show()
```




```
report = classification_report(
    y_test,
    predicted_labels,
    target_names=[str(i) for i in range(10)]
)

print(report)
```

```
              precision    recall  f1-score   support

           0       1.00      0.99      1.00       980
           1       1.00      0.99      1.00      1135
           2       0.99      0.98      0.99      1032
           3       0.99      0.99      0.99      1010
           4       0.99      1.00      0.99       982
           5       0.98      0.99      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      1.00      0.99      1028
           8       0.99      0.99      0.99       974
           9       1.00      0.98      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```
incorrect = np.where(predicted_labels != y_test)[0]

print("Number of incorrectly classified images:", len(incorrect))
```

```
Number of incorrectly classified images: 92
```

```
plt.figure(figsize=(12, 6))

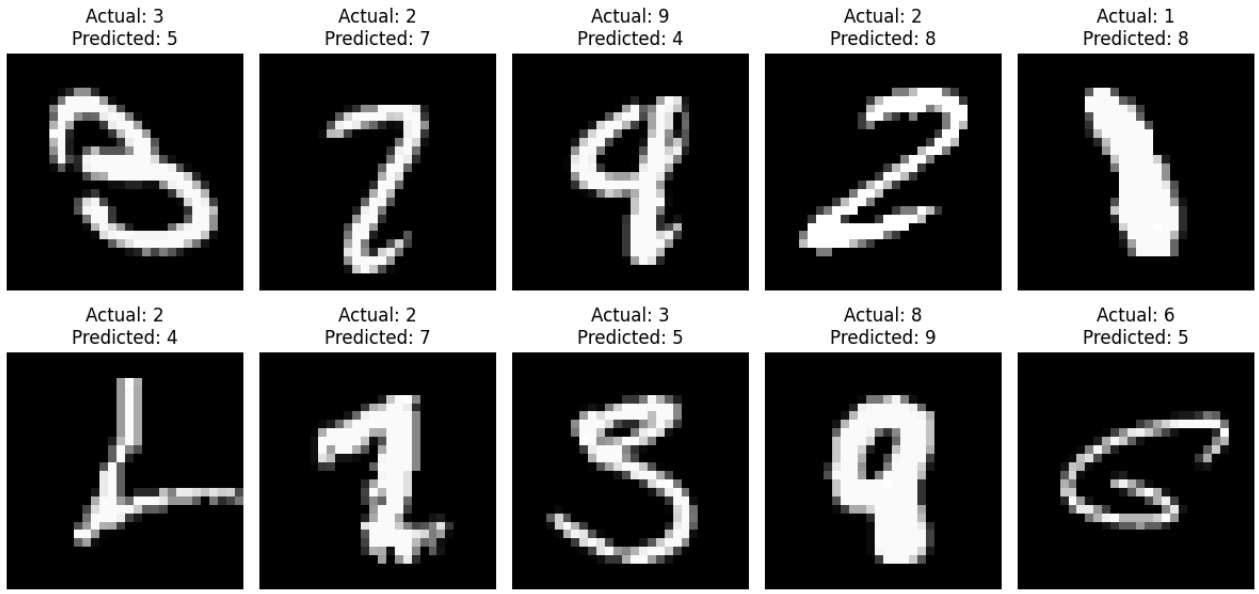
for i in range(min(10, len(incorrect))):
    index = incorrect[i]

    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[index].squeeze(), cmap="gray")

    plt.title(
        f"Actual: {y_test[index]}\n"
        f"Predicted: {predicted_labels[index]}"
    )
```

```
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```



Result:

The CNN model was successfully designed and trained using the MNIST handwritten digit dataset. The model learned important image features through convolutional and pooling layers and classified handwritten digits from 0 to 9. The trained model achieved high classification accuracy on the test dataset. Performance was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix.

Conclusion:

A Convolutional Neural Network was successfully implemented for handwritten digit classification using the MNIST dataset. The CNN automatically learned useful image features without requiring manually designed features. The results demonstrated that CNNs are highly effective for image classification tasks and can be applied to applications such as handwriting recognition, document processing, biometric systems, and other computer vision tasks.

Questions and Answers:

Q1. What is image classification? How does it differ from object detection and image segmentation?

Image classification is the process of assigning one or more class labels to an image.

For example, an MNIST image can be classified as the digit 7.

Classification: Determines what object or category is present. Object Detection: Identifies objects and their locations using bounding boxes. Image Segmentation: Classifies individual pixels into different regions or objects.

Therefore, classification gives the class, detection gives the class plus location, and segmentation provides pixel-level information.

Q2. Explain the architecture and working principle of a CNN.

A Convolutional Neural Network (CNN) is a deep learning model mainly used for processing images.

A typical CNN consists of:

Input layer Convolutional layers Activation functions Pooling layers Flatten layer Fully connected layers Output layer

The convolutional layers extract features such as edges, shapes, and patterns. Pooling reduces the spatial dimensions of the feature maps. The fully connected layers use the extracted features for classification. Finally, the output layer produces probabilities for the different classes.

Q3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

Convolutional Layers: Extract important features from an image using filters or kernels. Early layers may detect edges and simple patterns, while deeper layers can detect more complex patterns.

Pooling Layers: Reduce the spatial size of feature maps. This decreases computational requirements and helps the model focus on important features.

Fully Connected Layers: Use the extracted features to perform the final classification.

Q4. Why is image normalization performed before training a deep learning model?

Image normalization scales pixel values to a smaller range, commonly 0 to 1.

For example:

$$Normalized\ Pixel = \frac{Pixel}{255}$$

Instead of pixel values ranging from 0 to 255, the values become 0 to 1.

Normalization helps provide stable numerical values during training and can improve the efficiency and convergence of the neural network.

Q5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

ReLU (Rectified Linear Unit) is commonly used in hidden layers.

$$ReLU(x) = max(0, x)$$

It introduces non-linearity into the network and helps the CNN learn complex patterns.

Softmax is commonly used in the final classification layer. It converts the model's output values into probabilities for each class.

For MNIST, the Softmax layer produces probabilities for the 10 digits from 0 to 9.

Q6. What is a Confusion Matrix? How is it used to evaluate classification performance?

A confusion matrix is a table used to compare actual class labels with predicted class labels.

For a multi-class problem such as MNIST, each row represents the actual class and each column represents the predicted class.

Correct predictions appear along the main diagonal.

It helps identify:

Correct classifications Incorrect classifications Which classes are confused with each other

For example, if many actual 5s are predicted as 3s, the confusion matrix will show this error.

Q7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

Accuracy is the proportion of all predictions that are correct.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision measures how many predicted positive samples are actually positive.

$$Precision = \frac{TP}{TP + FP}$$

Recall measures how many actual positive samples were correctly detected.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score is the harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

For example, in a digit classification problem, precision for digit 8 indicates how many images predicted as 8 were actually 8, while recall for 8 indicates how many actual 8 images were correctly identified.

Q8. Compare traditional feature-based image classification methods such as SIFT/HOG + SVM with CNN-based classification. Feature SIFT/HOG + SVM CNN Feature extraction Handcrafted Automatically learned Classification Usually separate SVM classifier Integrated into network Feature engineering Required Minimal Large datasets Less suitable Highly suitable Computational requirement Generally lower Generally higher Complex image patterns Limited compared with deep CNNs Can learn complex patterns

Traditional methods depend on manually designed features, while CNNs automatically learn useful features from training data.

Q9. Mention five real-world applications of CNN-based image classification.

Five applications are:

Handwritten character recognition Face recognition Medical image classification Document and character recognition Product or object image classification

Other applications include defect detection, biometric authentication, agriculture, and satellite-image analysis.

Q10. How can Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve performance?

Data Augmentation: Creates modified versions of training images using operations such as rotation, shifting, cropping, and flipping. It increases data variety and can reduce overfitting.

Transfer Learning: Uses a model that has already learned useful features from a large dataset and adapts it to a new task. This can reduce training requirements and improve performance when the new dataset is limited.